

LLMs4OM — Code Repository Map

Navigation guide for revision · Master's Thesis 2025–2026 · KU Leuven

1. What This Is

This repo is a fork and extension of the LLMs4OM framework for ontology matching. It evaluates 7 retrieval models and 8+ LLMs on the OAEI 2024 Bio-ML track, and introduces a new **listwise ranking pipeline** with Reciprocal Rank Fusion (RRF) as the thesis contribution. All experiments run on the VSC HPC cluster (NVIDIA H100/V100 GPUs, SLURM).

2. Most Efficient Revision Order (Bottom-Up Data Flow)

Read leaf files before the orchestrator — each file is self-contained, and `om_pipeline.py` becomes trivial last.

Step	File(s)	Why This Order
1	ontology/bioml.py base/dataset.py	Defines task_owl — the dict every other file consumes
2	encoder/encoders.py encoder/rag.py	Defines encoded_inputs — tiny files, fast read
3	retrieval/retrieval.py retrieval/models.py	Self-contained, clean — builds confidence
4	rag/dataset.py	Just prompt templates — 5-minute read
5	rag/rag.py	Pairwise core: yes/no logit extraction, RAG class
6	rag/rag_listwise.py rag/dataset_listwise.py	Thesis contribution — read after pairwise
7	postprocess/process.py	Makes full sense once you know what RAG outputs
8	evaluation/evaluator.py evaluation/metrics.py	Trivial by this point — P/R/F1
9	pipeline/om_pipeline.py	Now it reads as 'just a loop calling all the above'
10	experiments/experiments.ipynb	Read last — all thesis analysis, tables, plots

3. Top-Level File Map (Repo Root)

File / Dir	Role
ontomap/	Main Python package — all core logic lives here
pipeline_rag.py	Run pairwise RAG pipeline over all Bio-ML tasks
pipeline_retrieval.py	Run retrieval-only pipeline
pipeline_listwise.py	Run listwise RAG pipeline
pipeline_full_*.py	Legacy per-model scripts from original LLMs4OM
pipeline_test.py / V2	Quick dev/test runners
test_rag.py	Interactive test for a single RAG model + dataset
test_rag_listwise.py	Listwise test — shows per-source ranking table with LLM output
test_retrieval.py	Interactive test for retrieval only

File / Dir	Role
test_llm*.py	LLM-specific unit tests
sweep_threshold_Qwen34_Qwen359B.py	Ssim threshold sweep (0.0→0.95), outputs CSV
sweep_rrf_weights.py	RRF weight (ir_weight, llm_weight) grid sweep
sweep_rrf_k.py	RRF k parameter sweep
print_rag_results.py	Print/summarize stored JSON results
*.batch	SLURM job scripts for HPC pipeline runs
experiments/	Notebooks, result CSVs, LaTeX tables, plots
datasets/	OWL files + pre-parsed om.json per task
scripts/	Utility: parse OWL→json, generate LaTeX, build embeddings
docs/	LLMs4OM_Supplementary_Material.pdf

4. The ontomap/ Package — Module by Module

4.1 base/ — Abstract Foundations

File	Key Class	What It Does
base/model.py	BaseOMModel	Abstract base: generate(input_data) + path attribute
base/configs.py	BaseConfig	All hyperparams. get_args() registers CLI args + model config dicts. vars(config)[
base/ontology.py	BaseOntologyParser	OWL parsing via owlready2: extracts {name, iri, label, childrens, parents, synonym
base/dataset.py	OMDataset, BioMLOMDataset	ABC dataset. collect()=parse OWL, load_from_json()=fast cached path
base/encoder.py	BaseEncoder	Abstract encoder with __call__(**kwargs)

4.2 ontology/ — Dataset Task Definitions

File	Key Classes	Notes
ontology/__init__.py	ontology_matching dict	GATE for which tasks run in pipeline. Only NCITDiseaseOMDataset
ontology/biomi.py	BioOntology, BioMLOMDataset, NCITDiseaseOMDataset, OMIMOMDataset, SNO_MED	Also BioOMDataset, TOMIMOMDataset, OMIMOMDataset, SNO_MED, FMABody
anatomy/biodiv/food/mse/phenotype.py	Phenotype	Other OAEI tracks — inactive, legacy from original LLMs4OM

4.3 encoder/ — Concept Representation

Encoders do NOT embed. They build the encoded_inputs dict: pointers to text format and prompt template.

File	Key Classes	Approach
encoder/__init__.py	EncoderCatalog dict	Registry: approach → encoder_id → class
encoder/encoders.py	RAGEncoder	__call__ builds IRI→index lookups, stores retrieval+LLM encoder refere
encoder/lightweight.py	IRILabel*LightweightEncoder (4 variants)	Label / Label+Parents / Label+Children / Label+Desc
encoder/rag.py	IRILabel*RAGEncoder (3 base + 3 Instruct + Pointwise)	Pointwise which RAGDataset class to use for prompting
encoder/fewshot.py	IRILabel*FewShotEncoder	Few-shot approach encoders
encoder/naivconvoaei.py	IRILabel*NaiveEncoder	Legacy naive approach

4.4 ontology_matchers/ — All Model Implementations

ontology_matchers/_init_.py → **MatcherCatalog**: maps approach → model_id → class. This is what the pipeline uses to instantiate models. To add a model: implement class, import here, add to dict.

retrieval/ — *Embedding-Based Retrieval*

File	Key Classes	Notes
retrieval/retrieval.py	Retrieval (base), BiEncoderRetrieval, MLRetrieval, BiEncoderRetrieval	BiEncoderRetrieval.generate() = GPU batch cosine sim (fast). Base Retrieval.generate() = CPU batch cosine sim (slow)
retrieval/models.py	BERTRetrieval, SpecterBERTRetrieval, OpenAIEmbeddingRetrieval, Qwen2EmbeddingRetrieval, Qwen2.5EmbeddingRetrieval, Qwen2.5EmbeddingRetrieval, Qwen2.5EmbeddingRetrieval	Qwen2.5EmbeddingRetrieval = Qwen2.5EmbeddingRetrieval (fast)
Output: [{source: iri, target-cands: [iri1..5], score-cands: [0.92..0.60]}, ...]		

rag/ — *RAG Pipeline (Thesis Core)*

File	Key Classes	What It Does
rag/rag.py	RAGBasedDecoderLLMArch	Adds yes/no logit scoring. ANSWER_SET → token IDs → softmax → confidence
rag/rag.py	RAGBasedInstructDecoderLLMArch	Subclass — adds chat template wrapping in tokenize()
rag/rag.py	RAG	Combines Retrieval+LLM. generate() calls ir_generate() then llm_generate()
rag/models.py	LLaMA3BertRAG, Gemma2_9BBertRAG, Qwen2.5BertRAG, Qwen2.5BertRAG, Qwen2.5BertRAG, Qwen2.5BertRAG	Each uses its own tokenizer and model. LLaMA3BertRAG uses LLaMA3 tokenizer and model. Qwen2.5BertRAG uses Qwen2.5 tokenizer and model. Qwen2.5BertRAG uses Qwen2.5 tokenizer and model. Qwen2.5BertRAG uses Qwen2.5 tokenizer and model.
rag/dataset.py	LabelRAGDataset, LabelParentRAGDataset, LabelChildRAGDataset	LabelRAGDataset fills template with concatenated source and target-cands. LabelParentRAGDataset fills template with concatenated source and target-cands. LabelChildRAGDataset fills template with concatenated source and target-cands.
rag/dataset_listwise.py	LabelListwiseRAGDataset, LabelParentListwiseRAGDataset, LabelChildListwiseRAGDataset	LabelListwiseRAGDataset fills template with concatenated source and target-cands. LabelParentListwiseRAGDataset fills template with concatenated source and target-cands. LabelChildListwiseRAGDataset fills template with concatenated source and target-cands.
rag/rag_listwise.py	RAGBasedListwiseLLMArch, ListwiseRAGBasedLLMArch	Deparse_ranking() text generation instead of logit scoring. parse_ranking()

llm/ — *Base LLM Architecture*

File	Key Classes	Notes
llm/llm.py	LLaMA2DecoderLLMArch	Base for all decoder LLMs. Defines load_tokenizer(), load_model(), tokenize(), generate()
llm/models.py	FlanT5XL/XXL, LLaMA7B/13B, etc.	Original LLMs4OM models — legacy, not used in thesis experiments

4.5 postprocess/ — *Filtering to Final Alignments*

File: postprocess/process.py

Function	When Used	What It Does
eval_preprocess_ir_outputs()	Retrieval-only eval	Deduplicates IR output, keeps pairs with score > 0
preprocess_ir_outputs()	Before LLM in pairwise RAG	Flattens {source, target-cands[], score-cands[]} → flat {source, target, score} pairs
postprocess_hybrid()	Pairwise RAG	Builds source×target matrix; one-to-one enforcement; filters by ir_score ≥ 0.9 AND llm_score ≥ 0.9
apply_rrf()	Listwise	ir_weight/(k+ir_rank+1) + llm_weight/(k+llm_rank+1)
postprocess_listwise()	Listwise RAG	RRF per pair → column pass → row pass → filter by ir_score ≥ 0.9
postprocess_heuristic()	Legacy	Original LLMs4OM confidence-ratio approach — not used in thesis

Key finding: ir_score_threshold=0.9 is the dominant filter — not the LLM score.

4.6 evaluation/ — *Metrics*

File	Function	Notes
evaluation/metrics.py	evaluation_report()	Computes P, R, F1 by comparing predicted pairs against reference pairs
evaluation/evaluator.py	evaluator()	Routes by track (bio-ml → {full, test, train} F1)
evaluation/evaluator.py	evaluator_module()	Called by pipeline second pass; applies correct postprocessing per approach then computes metrics

4.7 pipeline/ — *Top-Level Orchestrator*

File: `pipeline/om_pipeline.py` → `OMPipelines` class

Main loop: for model in `MatcherCatalog` × for task in `ontology_matching` × for encoder in `EncoderCatalog`

First pass (`do_evaluation=False`): generate + write JSON to `experiments/outputs/`

Second pass (`do_evaluation=True`): read JSON → `evaluator_module()` → write F1 back to JSON

5. Class Inheritance Trees

Retrieval

`BaseOMModel` → `Retrieval` → `BiEncoderRetrieval` → `BERT` / `Specter` / `Qwen3-0.6B` / `Qwen3-4B` / `Gemma300M` / `Nemotron` / `NVEmbed`

→ `MLRetrieval` → `SVMBert`

→ `TFIDFRetrieval`

→ `BM25Retrieval`

LLM / RAG (Pairwise)

`LLaMA2DecoderLLMArch` (`llm/llm.py`) ← base for all decoders

■■■ `RAGBasedDecoderLLMArch` (`rag/rag.py`) ← yes/no logit extraction

■■■ `LLaMA3DecoderLM`, `MistralNemoDecoderLM`, `Qwen25DecoderLM`,

■ `Qwen35_9BDecoderLM`, `Gemma2_9BDecoderLM`, `Gemma4_26B_A4BDecoderLM`, `Mamba3B`

■■■ `RAGBasedInstructDecoderLLMArch` ← chat template in `tokenize()`

■■■ `LLaMA3InstructDecoderLM` (system prompt in `tokenize()`)

`RAG` (`rag/rag.py`) ← `Retrieval` + LLM orchestrator

■■■ `LLaMA3BerTRAG`, `Qwen35_9BQwen34BRAG`, `Gemma4_26B_A4BBerTRAG`, etc.

■■■ `ListwiseRAG` (`rag_listwise.py`) ← overrides `llm_generate()`

■■■ `LLaMA3ListwiseBerTRAG`

■■■ `Gemma4ListwiseBerTRAG`

■■■ `Qwen35_9BListwiseBerTRAG`

LLM / RAG (Listwise — Thesis Contribution)

`RAGBasedListwiseLLMArch` ← text generation, no logit scoring

■■■ `RAGBasedListwiseInstructLLMArch` ← adds chat template

■ ■■■ `LLaMA3ListwiseDecoderLM`

■■■ `Gemma4ListwiseDecoderLM` (`enable_thinking=False`)

■■■ `Qwen35_9BListwiseDecoderLM` (`enable_thinking=False`)

6. The 4 Registry Files — Always Touch These When Adding Something

File	Dict Name	Edit When
<code>ontomap/ontology/__init__.py</code>	<code>ontology_matching</code>	Adding/enabling a new Bio-ML task
<code>ontomap/ontology_matchers/__init__.py</code>	<code>MatcherCatalog</code>	Adding a new model class
<code>ontomap/encoder/__init__.py</code>	<code>EncoderCatalog</code>	Adding a new encoder variant
<code>ontomap/base/configs.py</code>	<code>rag_icv_models</code> / <code>retriever_models</code>	Adding new model config registration

7. Experiments & Results

File / Dir	Contains
experiments/experiments.ipynb	ALL thesis analysis: tables, plots, comparisons — single source of truth
experiments/results/retrieval-model-results.csv	Retrieval-only F1 per model per task
experiments/results/rag-hybrid-model-results.csv	Pairwise RAG results
experiments/results/retrieval+lightweight+rag-hybrid-model-results.csv	Combined results
experiments/outputs/	Raw JSON outputs per model/task/encoder run
experiments/tables/*.tex	LaTeX tables per track per approach
experiments/visualizations/	PNG plots and HTML visualization
experiments/threshold_sweep_Qwen35_9BQwen34BRAG.csv	Threshold sweep CSV

8. End-to-End Checklist for Adding a New Model

Step	File	Action
1	rag/models.py (or rag_listwise.py)	Implement LLM class: set path, override load_tokenizer(), load_model()
2	rag/models.py	Implement RAG combo: class MyModelBertRAG(RAG): Retrieval=X; LLM=Y
3	ontology_matchers/__init__.py	Import + add to MatcherCatalog['rag']['MyModelBertRAG'] = MyModelBertRAG
4	base/configs.py	Add 'MyModelBertRAG' to rag_icv_models list
5	pipeline_rag.py	Set models-to-consider: ['MyModelBertRAG'] and run

For a retrieval model: step 1 in retrieval/models.py, step 3 in MatcherCatalog['retrieval'], step 4 in retriever_models.